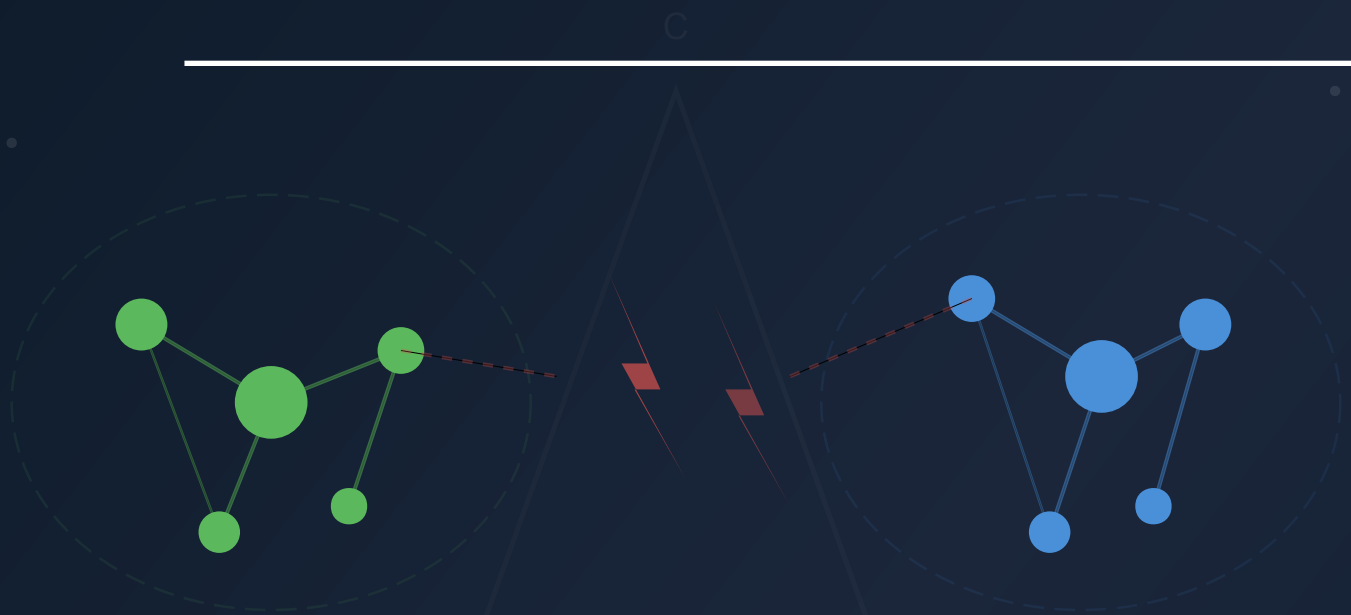
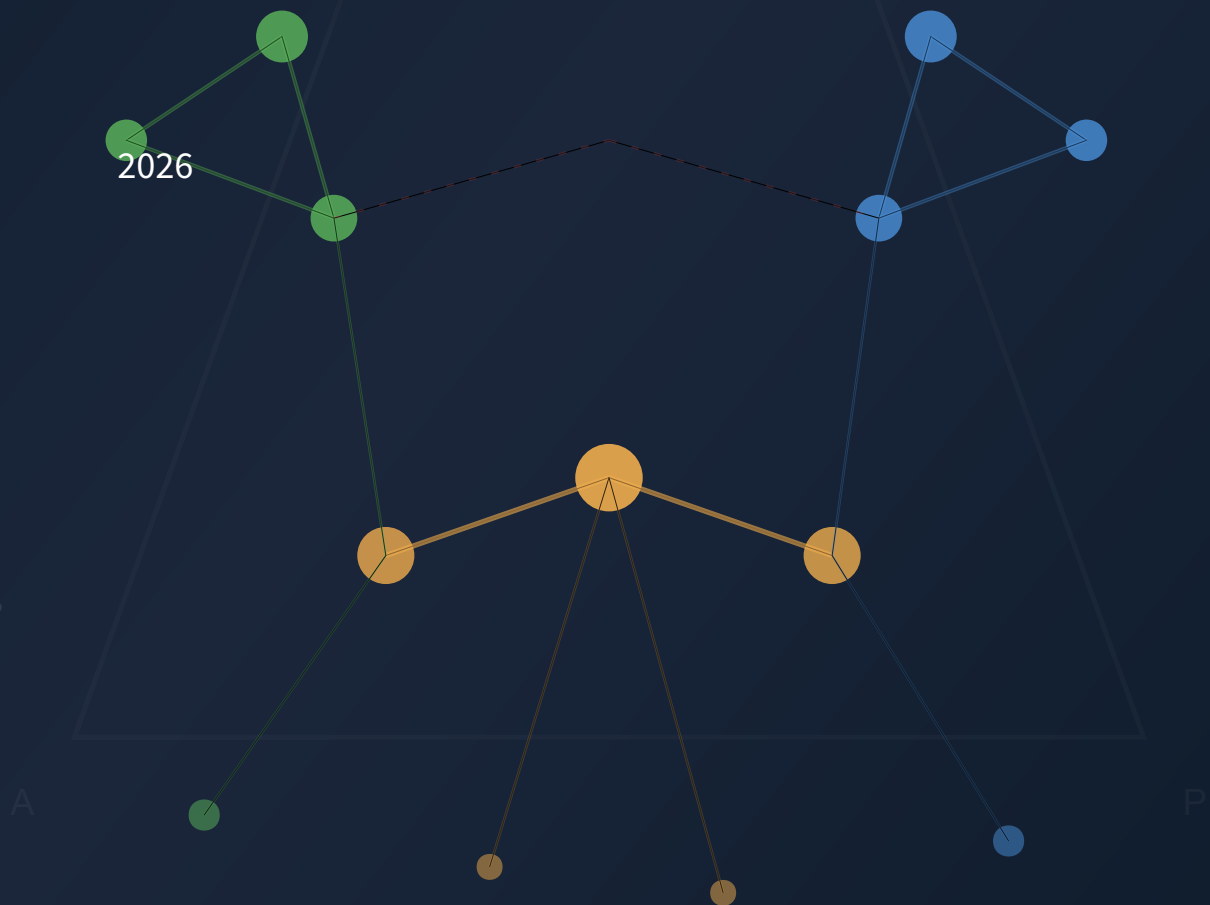




Distributed Systems Fundamentals

Week 3 - Course Notes



Contents

1	Week 3: Distributed Systems Fundamentals	2
1.1	The Story So Far	2
1.2	What We'll Discover	2
1.3	Class 3.1: The Big Three — Reliability, Scalability, Maintainability	3
1.3.1	Setting the Stage	3
1.3.2	The Core Idea	3
1.3.3	Real-World Application	5
1.3.4	Objective Check: O3.1	6
1.4	Class 3.2: The 8 Fallacies of Distributed Computing	7
1.4.1	Bridge from Class 3.1	7
1.4.2	The Core Idea	7
1.4.3	Going Deeper	8
1.4.4	Real-World Application	11
1.4.5	Objective Check: O3.2	11
1.5	Class 3.3: CAP Theorem and Big Data Context	11
1.5.1	Bridge from Class 3.2	11
1.5.2	The Core Idea	12
1.5.3	Going Deeper	12
1.5.4	Big Data Context: Why Distribution Became Necessary	14
1.5.5	Real-World Application	15
1.5.6	Objective Check: O3.3	15
1.6	Bringing It Together	16
1.6.1	The Week's Journey	16
1.6.2	Key Terms	16
1.6.3	Common Misconceptions	17
1.7	Looking Ahead	17
1.7.1	Connection to Next Week	17
1.7.2	Your Startup Application	18
1.8	Objective Verification Checklist	18
1.8.1	Diagram Audit	19
1.8.2	Connection Audit	19

1 Week 3: Distributed Systems Fundamentals

1.1 The Story So Far

In Week 1, we confronted a sobering reality: traditional infrastructure cannot scale gracefully. We traced an application's evolution from a simple static page to a database-backed system, watching as each new layer of complexity introduced new failure points. The provisioning dilemma became clear—over-provision and waste money, under-provision and lose customers.

Week 2 brought us REST and stateless design. We learned that separating state from computation enables horizontal scaling. JSON became our lingua franca. Git showed us how distributed collaboration is possible without a central authority. Yet one question lingered: **why is building distributed systems so fundamentally difficult?**

This week, we confront that question directly. We will discover that the challenges of distribution are not merely technical inconveniences—they are mathematical certainties. The properties we desire (reliability, scalability, maintainability) exist in tension with each other. The assumptions we make about networks are systematically false. And when we connect multiple machines, we face an impossible choice encoded in the CAP theorem.

By the end of this week, you will possess a diagnostic framework for evaluating any distributed system. More importantly, you will understand *why* certain trade-offs are unavoidable—knowledge that transforms you from a programmer who builds systems to an engineer who designs them.

1.2 What We'll Discover

By the end of this week, you will be able to:

Objective ID	Class	Objective
O3.1	3.1	Define reliability, scalability, and maintainability; distinguish faults from failures; identify which quality is compromised in a given scenario
O3.2	3.2	List all 8 Fallacies of Distributed Computing; explain why each is false; predict failure modes when a fallacy is assumed true
O3.3	3.3	State the CAP theorem; classify a system as CP or AP; justify trade-off choice for a given use case

1.3 Class 3.1: The Big Three — Reliability, Scalability, Maintainability

1.3.1 Setting the Stage

Consider two systems. System A processes 10,000 requests per second with 99.9% uptime and was built by a team of three engineers over six months. System B also processes 10,000 requests per second with 99.9% uptime—but it took 15 engineers two years to build, and every small change requires a two-week deployment cycle.

Which system is better?

The answer depends on what you value. System A might be a brilliant but fragile creation that only its original creators understand. System B might be an over-engineered monument to committee design. Or perhaps System A is a well-designed marvel, while System B suffers from accumulated technical debt.

This is precisely why Martin Kleppmann, in his seminal work *Designing Data-Intensive Applications*, argues that we need a vocabulary for discussing system quality. Without precise terms, we cannot compare designs, justify trade-offs, or communicate requirements. He proposes three fundamental qualities that every data-intensive application must address: **Reliability**, **Scalability**, and **Maintainability**.

These are not mere checkboxes. They are lenses through which every architectural decision should be evaluated.

1.3.2 The Core Idea

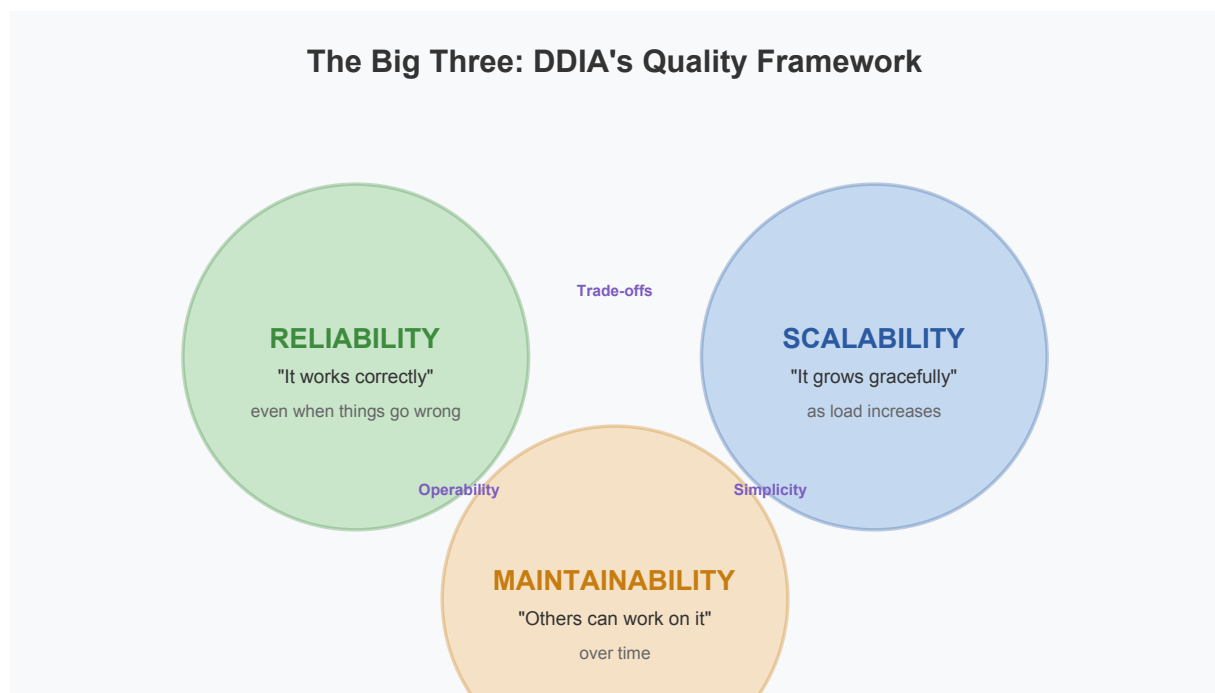


Figure 3.1: The Big Three — Reliability, Scalability, and Maintainability form the foundation of system quality evaluation.

1.3.2.1 Reliability: The System Works Correctly Despite Faults Reliability means the system continues to work correctly even when things go wrong. But what does “things going wrong” mean? Kleppmann distinguishes between **faults** and **failures**:

- A **fault** is a component deviating from its specification (a disk corrupts data, a process crashes, a network packet is lost)
- A **failure** is the system as a whole stopping to provide the required service

The goal is to build **fault-tolerant** systems—systems that prevent faults from causing failures. This distinction is crucial: we cannot prevent all faults (hardware will fail, humans will make mistakes), but we can design systems that tolerate them.

Kleppmann categorizes faults into three types:

Hardware Faults: Disk failures, RAM errors, power outages, network cable disconnections. Hard disks have a Mean Time To Failure (MTTF) of 10-50 years. In a storage cluster with 10,000 disks, you can expect one disk to die per day on average.

Software Faults: Bugs that cause crashes, runaway processes consuming shared resources, cascading failures where one component’s failure triggers failures in dependent components. These are often correlated—a bug triggered by unusual input will affect all machines running that software simultaneously.

Human Errors: Configuration mistakes, deploying untested code, accidentally deleting production data. Studies suggest that operator error causes 70-80% of outages.

```
1 # Example: Distinguishing faults from failures
2 class ReliableService:
3     def __init__(self, replicas=3):
4         self.replicas = replicas
5         self.healthy_replicas = replicas
6
7     def handle_disk_failure(self):
8         """A fault: one replica's disk fails"""
9         self.healthy_replicas -= 1
10        if self.healthy_replicas >= 2:
11            # System tolerates the fault - no failure
12            return "Fault tolerated, service continues"
13        else:
14            # Fault causes system failure
15            return "FAILURE: Cannot maintain consistency"
16
17    def is_operational(self):
18        """System fails only when too many faults accumulate"""
19        return self.healthy_replicas >= (self.replicas // 2 + 1)
```

1.3.2.2 Scalability: The System Handles Growth Scalability is not a binary attribute (“the system is scalable” or “is not scalable”). Rather, scalability asks: **“If the system grows in a particular way, what are our options for coping with that growth?”**

To discuss scalability, we need to describe load precisely. **Load parameters** are the numbers that describe current demands on the system:

- Requests per second to a web server
- Read-to-write ratio in a database
- Number of simultaneous active users in a chat application
- Cache hit rate

Once we can describe load, we can investigate performance:

- When load increases and resources stay fixed, how is performance affected?
- When load increases, how much must we increase resources to keep performance unchanged?

Performance metrics include:

- **Throughput:** Records processed per second (batch systems)
- **Response time:** Time between request and response (online systems)
- **Latency:** Time a request waits to be handled (technical distinction from response time)

For response times, averages are misleading. Use **percentiles**:

- **p50 (median):** Half of requests are faster, half are slower
- **p95:** 95% of requests are faster
- **p99:** 99% of requests are faster—the “tail latency” that affects your most engaged users

Amazon has observed that a 100ms increase in response time reduces sales by 1%. The customers with the slowest response times often have the most data—your most valuable customers.

1.3.2.3 Maintainability: The System Can Be Worked On The majority of software cost is not in initial development but in ongoing maintenance: fixing bugs, keeping systems operational, investigating failures, adapting to new platforms, adding new features, and repaying technical debt.

Kleppmann breaks maintainability into three principles:

Operability: Make it easy for operations teams to keep the system running smoothly. Good operability means good monitoring, automation support, documentation, predictable behavior, and avoiding dependency on individual machines.

Simplicity: Make it easy for new engineers to understand the system. Complexity accumulates accidentally through scope creep, tight coupling, tangled dependencies, and inconsistent naming. The best tool against complexity is **abstraction**—hiding implementation details behind clean interfaces.

Evolvability: Make it easy to change the system in the future. Requirements change. Agile methodologies provide frameworks for adaptation. System architecture must enable agility through modularity and loose coupling.

1.3.3 Real-World Application

Netflix provides a masterclass in the Big Three:

Reliability: Netflix pioneered Chaos Engineering with Chaos Monkey, which randomly terminates production instances. By deliberately injecting faults, they ensure their systems tolerate them. Their Simian Army expanded this to include Chaos Gorilla (simulates entire availability zone failure) and Chaos Kong (simulates entire region failure).

Scalability: Netflix handles over 15% of global internet traffic during peak hours. They describe load in terms of concurrent streams (>200 million subscribers, millions of concurrent streams). They use adaptive bitrate streaming—when network conditions degrade, video quality automatically reduces to maintain playback.

Maintainability: Netflix employs hundreds of engineering teams, each responsible for specific microservices. Clean APIs between services enable independent deployment. Extensive automation allows them to push code to production hundreds of times per day.

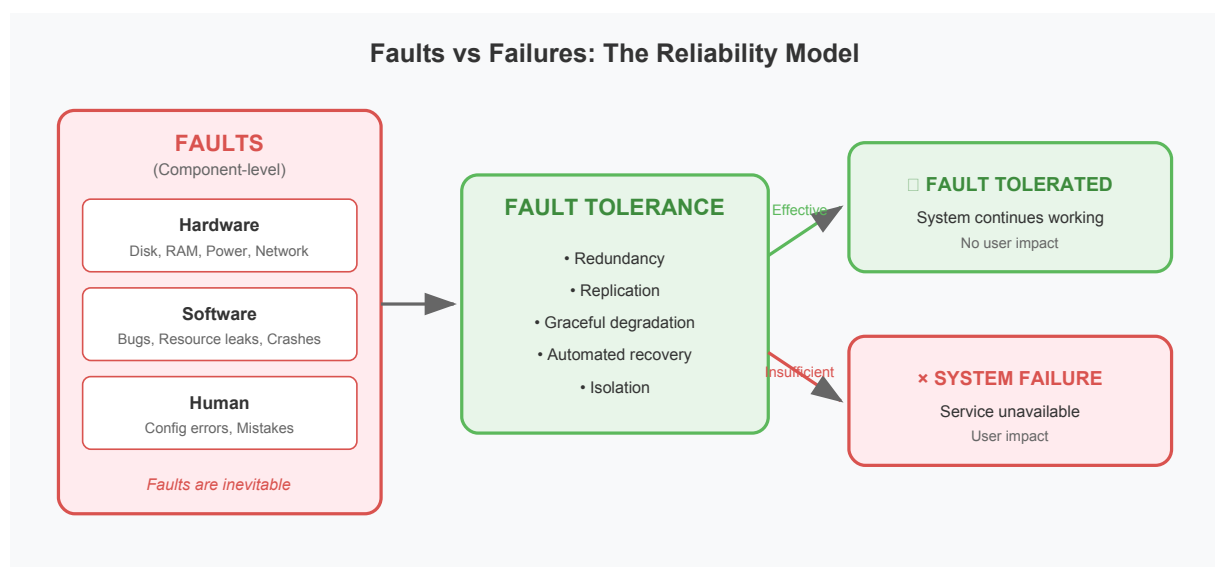


Figure 3.2: The path from faults to failures — Fault tolerance mechanisms determine whether component-level faults cause system-level failures.

1.3.4 Objective Check: O3.1

By now you should be able to define reliability, scalability, and maintainability; distinguish faults from failures; and identify which quality is compromised in a given scenario.

Quick test: A social media platform experiences a 10x traffic spike during a major event. Response times increase from 200ms to 3 seconds, but no requests fail. Which of the Big Three is compromised, and why?

Answer: Scalability is compromised—specifically, performance degrades under increased load. Reliability remains intact (no failures occur), and maintainability is not relevant to this runtime scenario.

1.4 Class 3.2: The 8 Fallacies of Distributed Computing

1.4.1 Bridge from Class 3.1

We have established that reliability, scalability, and maintainability are the qualities we seek. But why are these so difficult to achieve in distributed systems? Why can't we simply add more machines and expect linear improvements?

The answer lies in a set of false assumptions that nearly every engineer makes when first designing distributed systems. In 1994, Peter Deutsch, working at Sun Microsystems, catalogued seven of these assumptions. James Gosling later added an eighth. These **8 Fallacies of Distributed Computing** are not merely historical curiosities—they are traps that continue to ensnare even experienced engineers today.

Understanding these fallacies transforms how you think about distributed systems. Each false assumption leads to specific failure modes. Each failure mode demands specific mitigations.

1.4.2 The Core Idea

The 8 Fallacies are assumptions that programmers new to distributed applications invariably make. All are false. All cause problems and lead to system failures.

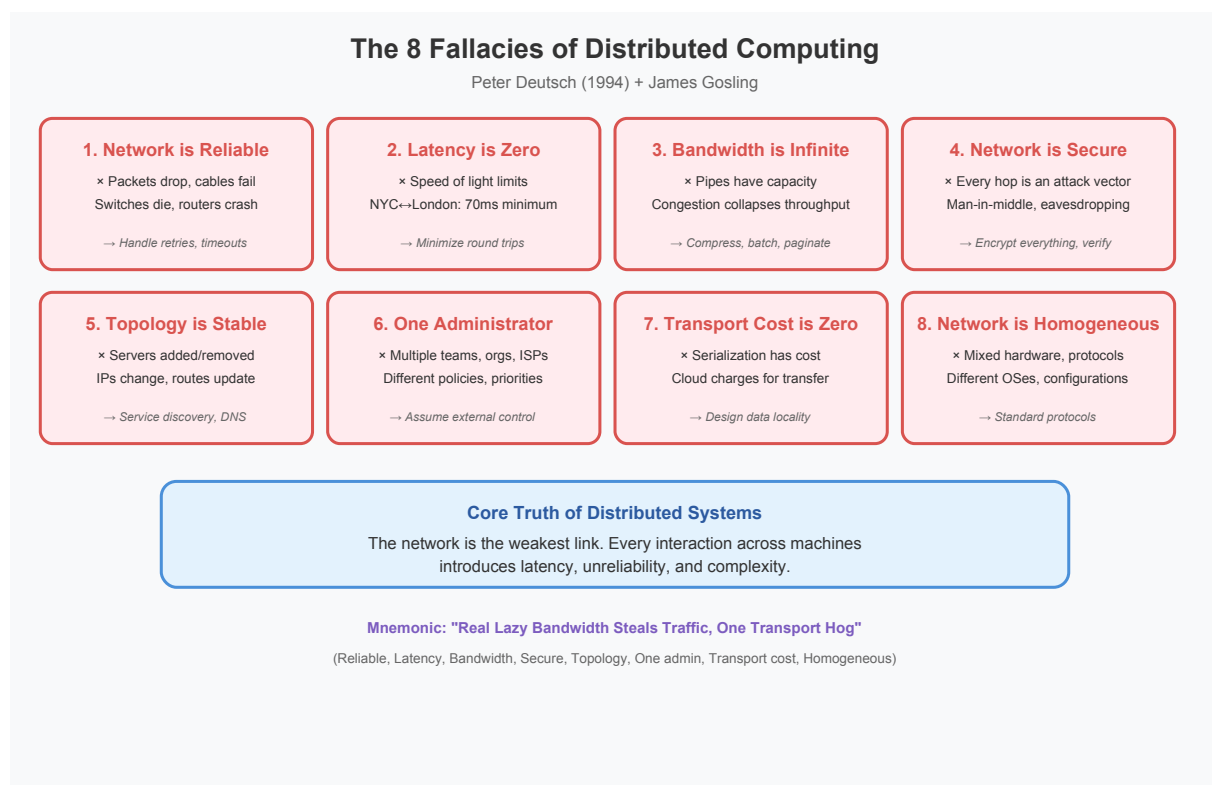


Figure 3.3: The 8 Fallacies of Distributed Computing — Each false assumption leads to specific failure modes that must be mitigated through explicit design.

1.4.3 Going Deeper

Let us examine each fallacy in detail, understanding why it is false and what failures it causes.

1.4.3.1 Fallacy 1: The Network is Reliable **The Assumption:** Network requests will always succeed. Messages sent will be received.

The Reality: Networks experience: - Packet loss due to congestion, buffer overflows, or interference - Hardware failures in switches, routers, and cables - Software bugs in network stacks - Configuration errors causing routing failures

The probability of a single request failing might be low (0.1%), but at scale, failures become certainties. If you make 10,000 requests per second, you will see ~10 failures per second.

Failure Modes: - Silent message loss (sender thinks message delivered, receiver never gets it) - Connection timeouts - Partial failures (request received, response lost)

Mitigations: - Implement retries with exponential backoff - Use acknowledgment protocols - Design for idempotency (same request can be safely repeated) - Implement circuit breakers to detect failing connections

```
1 import time
2 import random
3
4 def call_with_retry(func, max_retries=3, base_delay=1.0):
5     """
6     Implement exponential backoff with jitter
7     to handle network unreliability
8     """
9     for attempt in range(max_retries):
10        try:
11            return func()
12        except NetworkError as e:
13            if attempt == max_retries - 1:
14                raise # Final attempt failed
15
16            # Exponential backoff with jitter
17            delay = base_delay * (2 ** attempt)
18            jitter = random.uniform(0, delay * 0.1)
19            time.sleep(delay + jitter)
20
21        print(f"Retry {attempt + 1}/{max_retries} after {delay:.2f}s")
```

1.4.3.2 Fallacy 2: Latency is Zero **The Assumption:** Network calls are instantaneous, just like local function calls.

The Reality: - Speed of light in fiber: ~200,000 km/s - New York to London: 5,500 km → minimum 55ms round trip - Add routing, queuing, processing: real-world latency often 100-300ms

A local function call takes nanoseconds. A network call takes milliseconds. That's a factor of 1,000,000x difference.

Failure Modes: - “Chatty” protocols that make many round trips - Synchronous calls that block waiting for responses - Cascading timeouts

Mitigations: - Batch operations to reduce round trips - Use asynchronous communication - Cache frequently accessed data locally - Prefer data locality in architecture

1.4.3.3 Fallacy 3: Bandwidth is Infinite **The Assumption:** You can send unlimited amounts of data instantly.

The Reality: Networks have finite capacity. Shared links become congested. Bandwidth varies dramatically: - Local network: 1-100 Gbps - Internet backbone: Variable - Mobile connection: 1-100 Mbps - Satellite: 10-50 Mbps with high latency

Failure Modes: - Request timeouts during large transfers - Increased latency under congestion - Unfair resource consumption affecting other services

Mitigations: - Compress data before transmission - Implement pagination for large datasets - Use streaming for large transfers - Monitor bandwidth consumption

1.4.3.4 Fallacy 4: The Network is Secure **The Assumption:** Data transmitted is private and unmodified.

The Reality: Every network hop is an opportunity for: - Eavesdropping (reading data) - Man-in-the-middle attacks (modifying data) - Replay attacks (re-sending captured data) - Denial of service

Even “private” networks within a data center have been compromised.

Failure Modes: - Data breaches - Unauthorized access - Data corruption - Service disruption

Mitigations: - Encrypt all network traffic (TLS everywhere) - Authenticate all parties (mutual TLS) - Validate and sign messages - Implement defense in depth

1.4.3.5 Fallacy 5: Topology Doesn’t Change **The Assumption:** The network structure is fixed. IP addresses are permanent.

The Reality: - Servers are added, removed, upgraded - Cloud instances are ephemeral - Network paths change due to failures or optimization - IP addresses are reassigned

In cloud environments, you cannot assume any IP address will exist tomorrow.

Failure Modes: - Hardcoded addresses become invalid - Services become unreachable after topology changes - Stale connections to defunct servers

Mitigations: - Use service discovery mechanisms - DNS-based routing - Configuration through environment variables - Health checks to detect changes

1.4.3.6 Fallacy 6: There is One Administrator **The Assumption:** You control the entire network path.

The Reality: Your request traverses: - Your application servers (you control) - Your cloud provider's network (limited visibility) - Internet backbone providers (no control) - Destination's infrastructure (no control)

Each entity has different priorities, maintenance schedules, and security policies.

Failure Modes: - Unexpected maintenance windows - Incompatible policy changes - Blame-shifting during outages - Compliance violations

Mitigations: - Design for autonomy (don't depend on external uptime) - Implement circuit breakers for external services - Monitor third-party dependencies - Have fallback strategies

1.4.3.7 Fallacy 7: Transport Cost is Zero **The Assumption:** Sending data over the network is free.

The Reality: - Serialization/deserialization consumes CPU - Network hardware costs money - Cloud providers charge for data transfer (often significant) - Network operations consume power

AWS, for example, charges \$0.09/GB for data leaving a region. A chatty microservices architecture can generate surprising bills.

Failure Modes: - Unexpectedly high cloud bills - CPU exhaustion from serialization - Battery drain on mobile devices

Mitigations: - Monitor data transfer volumes - Design for data locality - Use efficient serialization formats (Protocol Buffers, MessagePack) - Cache aggressively

1.4.3.8 Fallacy 8: The Network is Homogeneous **The Assumption:** All systems use the same protocols, data formats, and conventions.

The Reality: - Different operating systems (Linux, Windows, macOS) - Different programming languages (Java, Python, Go) - Different byte orders (big-endian vs little-endian) - Different character encodings (UTF-8, Latin-1, etc.) - Different API versions

Failure Modes: - Data corruption from encoding mismatches - Protocol incompatibility - Silent data loss

Mitigations: - Use standard protocols (HTTP, gRPC) - Specify encodings explicitly - Version your APIs - Test across different platforms

1.4.4 Real-World Application

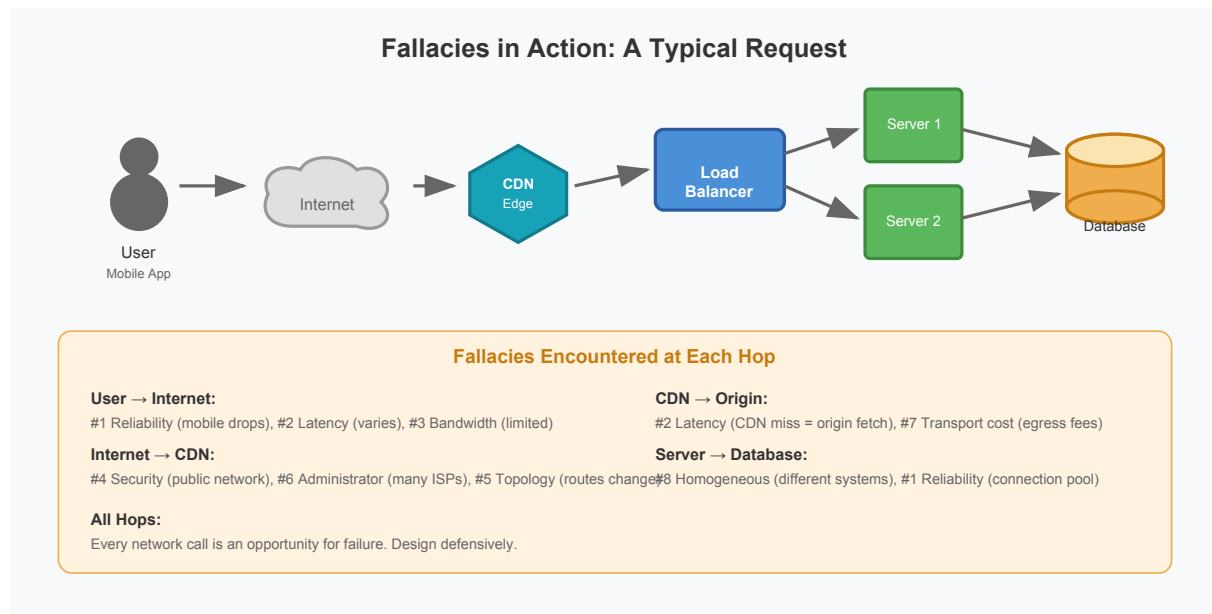


Figure 3.4: A single user request traverses multiple network boundaries, each encountering different fallacies. Robust systems must account for failures at every hop.

1.4.5 Objective Check: O3.2

By now you should be able to list all 8 Fallacies of Distributed Computing, explain why each is false, and predict failure modes when a fallacy is assumed true.

Quick test: A developer designs a system assuming all API calls complete within 50ms. When deployed globally, response times vary from 30ms to 300ms. Which fallacy did they violate, and what failure mode will occur?

Answer: They violated Fallacy #2 (Latency is Zero). Failure modes include: cascading timeouts (operations expecting 50ms time out), poor user experience for distant users, and potential thundering herd problems when operations take longer than expected.

1.5 Class 3.3: CAP Theorem and Big Data Context

1.5.1 Bridge from Class 3.2

The 8 Fallacies teach us that networks are fundamentally unreliable. But what happens when we try to build systems despite this unreliability? Can we have everything we want—perfect consistency, perfect availability, and perfect partition tolerance?

In 2000, Eric Brewer presented a conjecture that would become one of the most influential results in distributed systems. Two years later, Seth Gilbert and Nancy Lynch proved it mathematically. The **CAP theorem** states that we cannot have all three desirable properties simultaneously.

Understanding CAP is not merely academic. It guides fundamental architectural decisions. Should your banking system prioritize consistency or availability? What about a social media feed? The answer shapes everything from database choice to error handling strategy.

1.5.2 The Core Idea

The CAP theorem states: **In a distributed system, you can guarantee at most two of three properties: Consistency, Availability, and Partition Tolerance.**

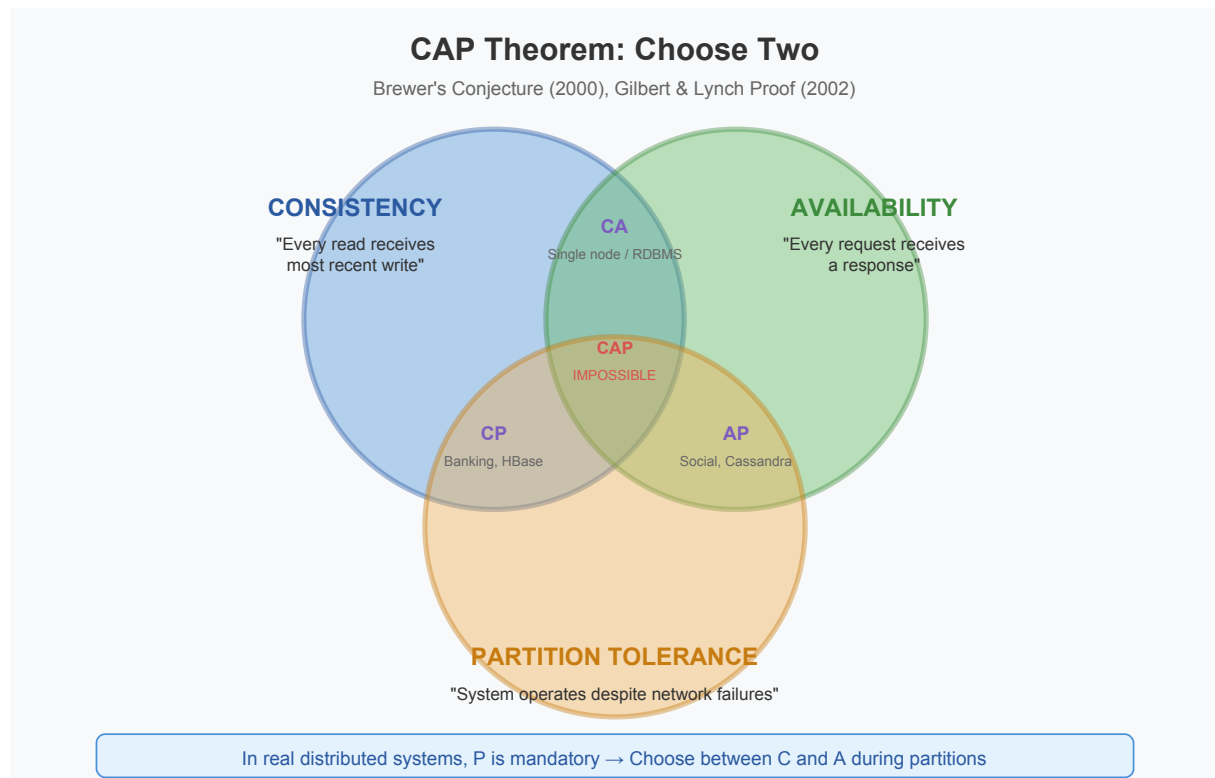


Figure 3.5: The CAP theorem visualized — In distributed systems, partitions are inevitable, forcing a choice between consistency and availability.

Let us define each property precisely:

Consistency (C): Every read receives the most recent write or an error. All nodes see the same data at the same time. If you write a value, any subsequent read will return that value.

Availability (A): Every request receives a (non-error) response, without guarantee that it contains the most recent write. The system is always operational and responsive.

Partition Tolerance (P): The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

1.5.3 Going Deeper

1.5.3.1 Why We Must Choose The key insight is that **network partitions are inevitable**. Hardware fails. Cables are cut. Routers crash. When a partition occurs, distributed nodes cannot communicate.

Consider a simple two-node system with a partition:

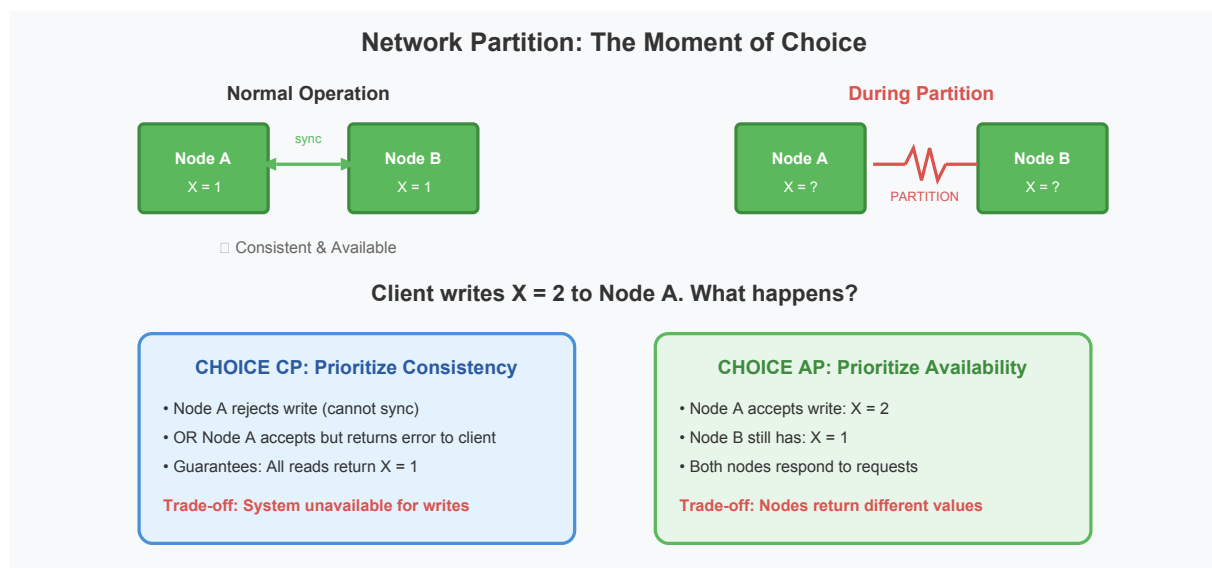


Figure 3.6: When a partition occurs, the system must choose: refuse to respond (maintain consistency) or respond with potentially stale data (maintain availability).

1.5.3.2 CP Systems: Consistency + Partition Tolerance CP systems sacrifice availability during partitions. When nodes cannot communicate, they may refuse to serve requests to ensure consistency.

Example: A banking system. When you transfer money, both accounts must be updated atomically. During a partition, the system refuses the transaction rather than risk inconsistency (money appearing in both accounts or neither).

Characteristics: - May return errors during partitions - Strong consistency guarantees - Suitable when correctness is paramount

Technologies: HBase, MongoDB (in certain configurations), Redis (in cluster mode), etcd, ZooKeeper

1.5.3.3 AP Systems: Availability + Partition Tolerance AP systems sacrifice consistency during partitions. All nodes continue to serve requests, but may return stale or conflicting data.

Example: A social media timeline. During a partition, you can still post and read posts, but you might not see your friend's most recent update immediately. Eventually, all nodes will synchronize.

Characteristics: - Always responsive - May return stale data - Requires conflict resolution strategies - Suitable when uptime matters more than perfect accuracy

Technologies: Cassandra, CouchDB, DynamoDB, Riak

1.5.3.4 The Reality: P is Not Optional In distributed systems, partitions will occur. Network cables are cut. Data centers lose connectivity. Therefore, the practical choice is between **CP** and **AP**.

This does not mean you always have reduced capability. During normal operation (no partition), you can have both consistency and availability. CAP only forces a choice during actual partitions.

```

1  class CAPAwareService:
2      """
3      Demonstrates CP vs AP behavior during partitions
4      """
5      def __init__(self, mode='CP'):
6          self.mode = mode # 'CP' or 'AP'
7          self.local_value = None
8          self.peer_reachable = True
9
10     def write(self, value):
11         if self.mode == 'CP':
12             # CP: Require peer acknowledgment
13             if not self.peer_reachable:
14                 raise UnavailableError(
15                     "Cannot write: partition detected. "
16                     "Refusing to risk inconsistency."
17                 )
18             # In real system: write to both, wait for acks
19             self.local_value = value
20             return "Write committed consistently"
21
22         else: # AP mode
23             # AP: Accept write locally, sync later
24             self.local_value = value
25             if not self.peer_reachable:
26                 return "Write accepted (will sync when partition heals)"
27             return "Write committed"
28
29     def read(self):
30         if self.mode == 'CP':
31             if not self.peer_reachable:
32                 raise UnavailableError(
33                     "Cannot guarantee consistent read during partition"
34                 )
35             return self.local_value
36
37         else: # AP mode
38             # Always return something, might be stale
39             return self.local_value

```

1.5.4 Big Data Context: Why Distribution Became Necessary

The CAP theorem gained practical urgency in the 2000s as data volumes exploded. Traditional single-node databases could not keep pace. Distribution was no longer optional—it was survival.

The Big Data Characteristics (3 V's):

Volume: Facebook processes 4+ petabytes of new data daily. Google processes 20+ petabytes. No single machine can store this.

Velocity: Twitter handles 400+ million tweets per day. Financial systems process millions of transactions per second. Real-time requirements demand parallel processing.

Variety: Structured (relational), semi-structured (JSON, XML), and unstructured (images, videos, logs) data must be handled together.

These characteristics drove the development of systems like Google's BigTable (2006), Amazon's Dynamo (2007), and Apache Hadoop/HDFS. Each made explicit CAP trade-offs:

- **BigTable/HBase:** CP orientation. Strong consistency for Google's indexing needs.
- **Dynamo/DynamoDB:** AP orientation. "Always writable" for Amazon's shopping cart.
- **HDFS:** CP orientation. Consistency for batch processing correctness.

Modern cloud databases often let you tune the trade-off: - DynamoDB offers "eventually consistent" and "strongly consistent" read options - Cassandra allows configurable consistency levels per operation - MongoDB supports different write concerns and read preferences

1.5.5 Real-World Application

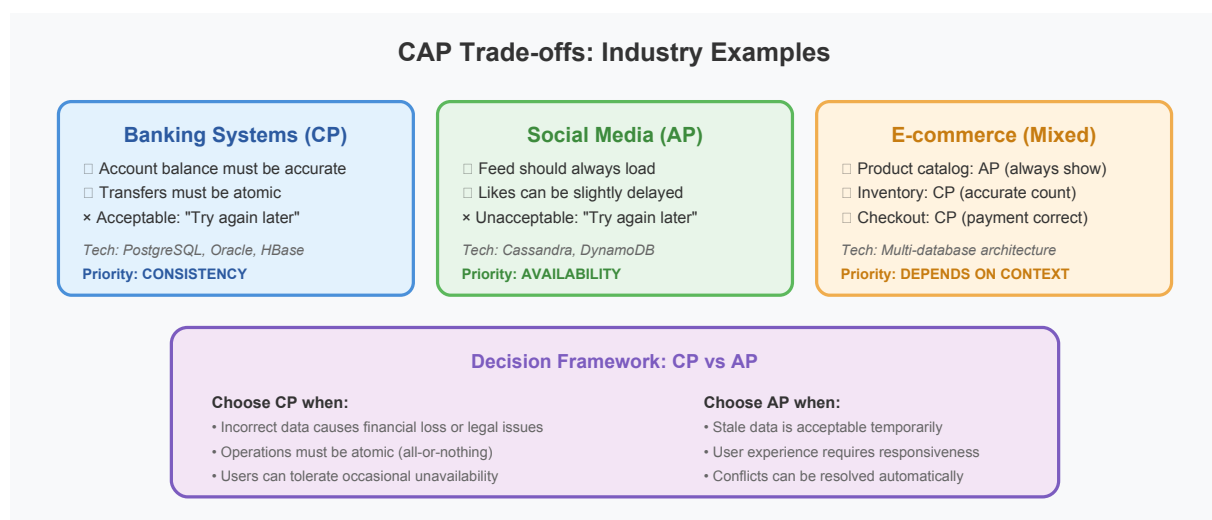


Figure 3.7: Real-world systems make deliberate CAP trade-offs based on their specific requirements. Many systems use different strategies for different features.

1.5.6 Objective Check: O3.3

By now you should be able to state the CAP theorem, classify a system as CP or AP, and justify trade-off choices for given use cases.

Quick test: A ride-sharing application needs to track driver locations in real-time. During a network partition, should the system: (a) Stop showing driver locations to ensure accuracy (CP) (b) Show potentially stale locations to keep the app functional (AP)

Answer: AP is appropriate. Showing a driver location that is 30 seconds old is far better than showing nothing. Users can tolerate slightly stale location data—the driver will arrive regardless. Availability keeps the service usable; perfect consistency is not critical for this feature.

1.6 Bringing It Together

1.6.1 The Week's Journey

We began this week asking why distributed systems are so difficult. Now we have answers.

Reliability, Scalability, and Maintainability (Class 3.1) gave us a vocabulary for quality. We learned to distinguish faults from failures, to describe load precisely with parameters and percentiles, and to value maintainability as much as performance. These are the goals we strive toward.

The 8 Fallacies (Class 3.2) explained why those goals are hard to achieve. Every assumption we naturally make about networks is wrong. Networks are unreliable, slow, limited, insecure, changing, fragmented, costly, and heterogeneous. Each false assumption leads to specific failure modes that must be explicitly mitigated.

The CAP Theorem (Class 3.3) revealed an impossibility result. When partitions occur—and they will—we cannot have both consistency and availability. We must choose, and that choice shapes our entire architecture.

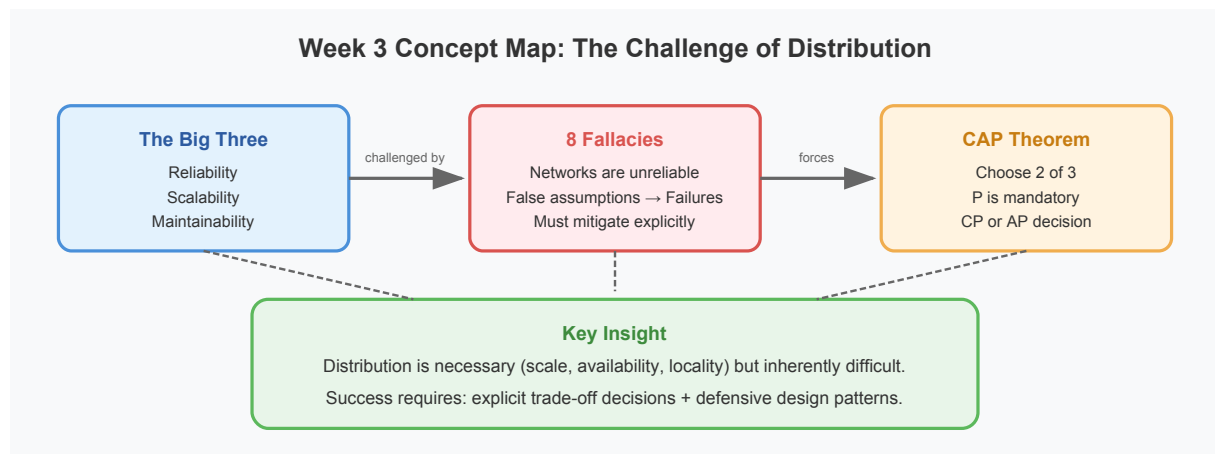


Figure 3.8: Week 3 integrates three perspectives on distributed systems: what we want (Big Three), why it's hard (Fallacies), and what's impossible (CAP).

1.6.2 Key Terms

Term	Definition	Example
Fault	Component deviating from specification	Disk corruption, process crash
Failure	System not providing required service	Website down, data loss
Fault Tolerance	Ability to continue operating despite faults	Running on multiple servers
Load Parameter	Metric describing system demand	Requests/second, active users
Percentile (p99)	Value below which 99% of observations fall	p99 latency = 500ms

Term	Definition	Example
Partition	Network failure preventing communication	Data center connectivity loss
Consistency (CAP)	All nodes see same data simultaneously	Bank balance identical everywhere
Availability (CAP)	Every request receives a response	System always operational
CP System	Prioritizes consistency during partitions	Banking, inventory systems
AP System	Prioritizes availability during partitions	Social feeds, caches
Eventual Consistency	All replicas converge eventually	Social media likes

1.6.3 Common Misconceptions

Misconception	Reality	Why It Matters
“Availability means 99.9% uptime”	CAP availability means ALL requests get responses, not a percentage	Understanding CAP requires precise definitions
“We should design for CAP”	P is given; design for CP or AP	Focus on the actual decision
“More replicas = more reliability”	Replicas must be coordinated; more replicas can mean more complexity	Complexity can reduce maintainability
“The 8 Fallacies are outdated”	Modern networks still exhibit all 8 problems	Cloud abstracts but doesn’t eliminate fundamentals
“Latency = response time”	Latency is waiting time; response time includes processing	Precise terminology prevents miscommunication

1.7 Looking Ahead

1.7.1 Connection to Next Week

We now understand the challenges of distributed systems. But understanding problems is not the same as solving them. How do we actually make decisions when faced with competing

requirements?

Week 4 introduces **System Design Trade-offs and Guidelines**. We will learn:

- The **7 System Design Concepts** that expand on DDIA's Big Three
- **4 Core Trade-offs** that appear in every design decision
- **5 Design Guidelines** that provide decision-making frameworks
- **SLI/SLO/SLA** frameworks for measuring and committing to quality
- **Day 0 → Day N thinking** for evolutionary architecture

The Big Three told us WHAT to optimize for. The Fallacies told us WHY it's hard. The trade-off framework will tell us HOW to choose.

1.7.2 Your Startup Application

For your startup project this week, analyze what breaks at scale:

Week 3 Task: Scale Scenario Analysis

1. Take your startup idea from Week 1
2. Imagine you suddenly have 10x your expected users
3. Answer these questions:
 - Which of the Big Three is most critical for your application? Why?
 - Which of the 8 Fallacies would hurt you most? How would you mitigate?
 - Would you choose CP or AP for your main feature? Justify with CAP reasoning.
 - What happens to your REST API (from Week 2) under 10x load?

Deliverable: A 1-page analysis addressing each question with specific, justified answers.

1.8 Objective Verification Checklist

Objective ID	Objective	Realized In Section	Diagrams	Word Count
O3.1	Define reliability, scalability, maintainability; distinguish faults from failures; identify which quality is compromised	Class 3.1: The Big Three	Fig 3.1, Fig 3.2	~1,100

Objective ID	Objective	Realized In Section	Diagrams	Word Count
O3.2	List all 8 fallacies; explain why each is false; predict failure modes	Class 3.2: 8 Fallacies	Fig 3.3, Fig 3.4	~1,200
O3.3	State CAP theorem; classify CP or AP; justify trade-off choice	Class 3.3: CAP Theorem	Fig 3.5, Fig 3.6, Fig 3.7	~1,100

1.8.1 Diagram Audit

Figure	Title	Components Used	Verified
3.1	The Big Three	Circles (custom), text labels	✓
3.2	Faults vs Failures	Rectangles, arrows, text	✓
3.3	8 Fallacies	Grid of rectangles, text	✓
3.4	Fallacies in Action	Server, Database, CDN, Cloud, User, arrows	✓
3.5	CAP Theorem	Circles (Venn), text labels	✓
3.6	Network Partition	Server, partition symbol, choice boxes	✓
3.7	Industry Examples	Rectangles with examples	✓
3.8	Week 3 Concept Map	Connected rectangles, arrows	✓

1.8.2 Connection Audit

From	To	Connection Type
Week 2 (Stateless design)	O3.1 (Scalability)	Statelessness enables horizontal scaling
O3.1 (Big Three)	O3.2 (Fallacies)	Fallacies explain why Big Three is hard
O3.2 (Fallacy 1: Reliability)	O3.3 (Partitions)	Unreliable networks cause partitions
O3.3 (CAP trade-offs)	Week 4 (Trade-offs framework)	Specific CAP choice → general trade-off thinking
